

AI Assisted Coding Assignment - 13

Chilukala Sai Architha(2303A52220) - Batch 37

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Instructions:

- Ask AI to locate repeated computations within the program.
- Encapsulate repeated logic into reusable functions.
- Verify that program output remains unchanged after refactoring.
- Add proper docstrings to all defined functions.

Sample Code:

```
length = 8
width = 4

print("Area:", length * width)

print("Perimeter:", 2 * (length + width))

length = 12
width = 6

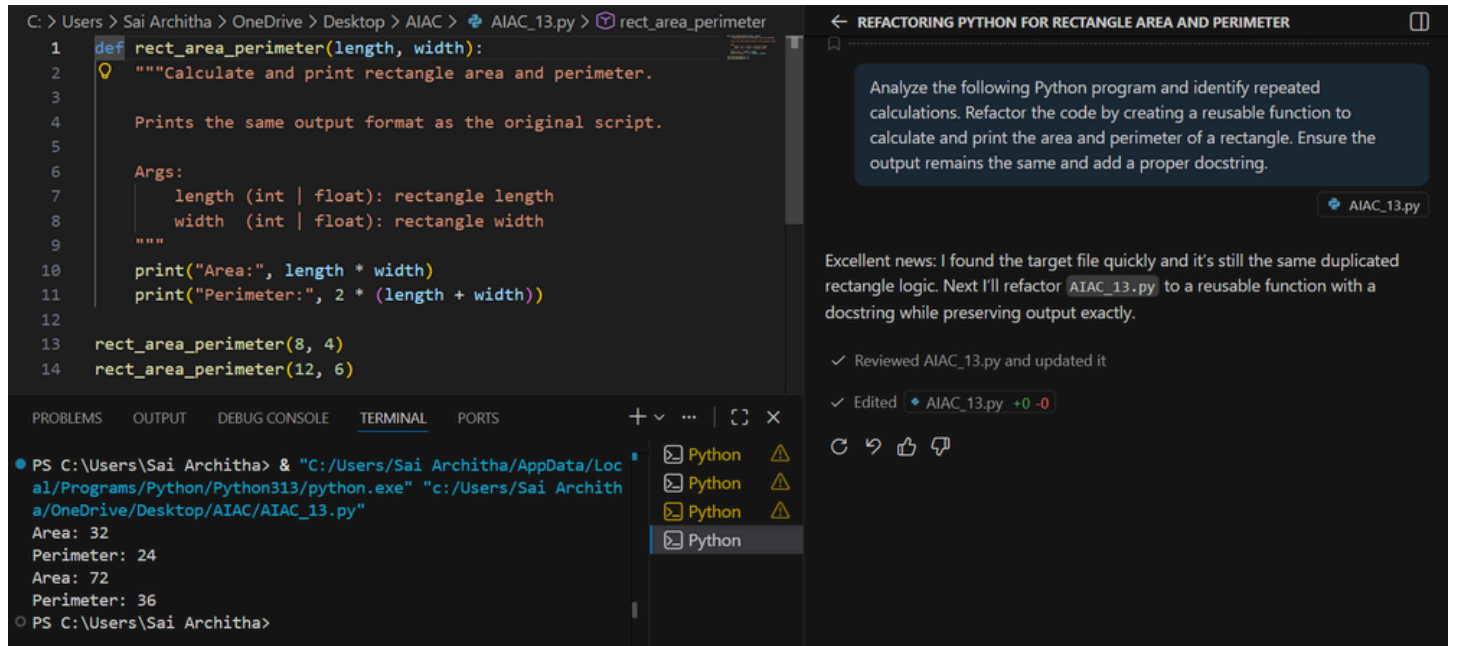
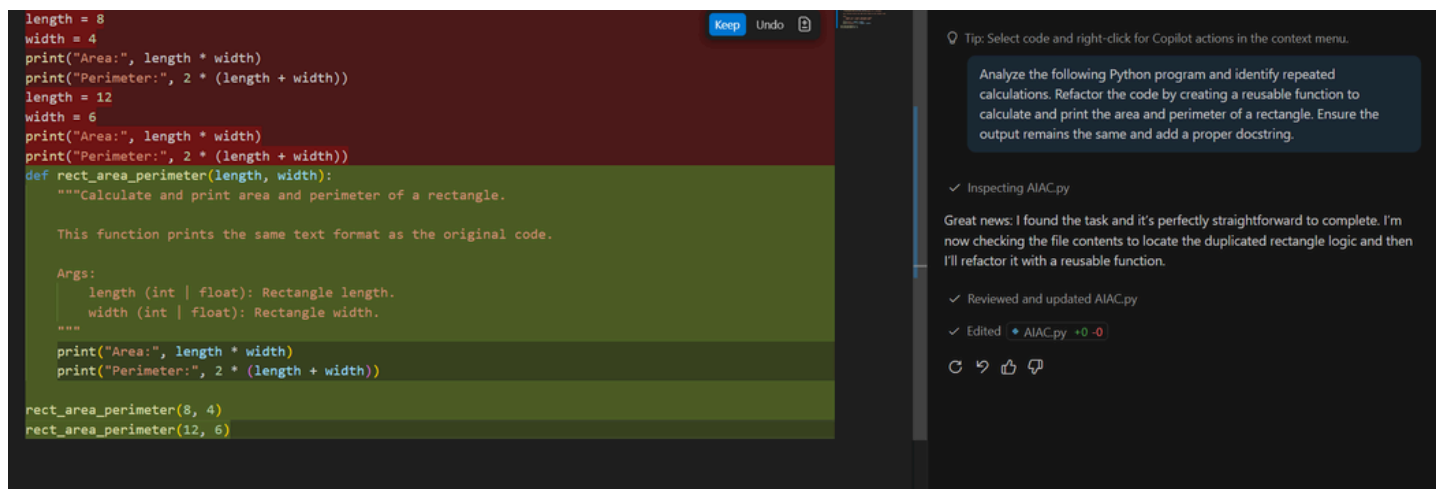
print("Area:", length * width)

print("Perimeter:", 2 * (length + width))
```

Expected Output:

A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

Prompt Used: Analyze the following Python program and identify repeated calculations. Refactor the code by creating a reusable function to calculate and print the area and perimeter of a rectangle. Ensure the output remains the same and add a proper docstring.



Explanation: The repeated calculations for area and perimeter were placed inside a reusable function called `rectangle_details()`. This improves code readability, reduces repetition, and makes the program easier to maintain.

Task Description -2 (Refactoring – Modularizing Inline calculations)

Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Instructions:

- Identify repeated calculation patterns in the code.
- Convert them into reusable functions.
- Ensure readability and proper documentation.

Sample Code:

amount = 1000

```

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

amount = 2000

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

```

Expected Output:

A modular Python program that uses a reusable function to compute final amounts for different inputs.

Prompt Used: Examine the Python code and identify repeated inline calculations related to discount and final amount. Refactor the program by creating a reusable function that calculates the final amount after a 10% discount. Improve readability and documentation.

```

> Users > Sai Architha > OneDrive > Desktop > AIAC > AIAC_13.py > ...
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
1 def calculate_final_amount(amount, discount_rate=0.10):
2     """Return final amount after applying a discount.
3
4     Args:
5         amount (int | float): Original amount before discount.
6         discount_rate (float): Discount fraction to apply (default 0.10)
7
8     Returns:
9         float: Amount after subtracting discount.
10    """
11    discount = amount * discount_rate
12    return amount - discount
13
14    final_amount = calculate_final_amount(1000)
15    print("Final Amount:", final_amount)
16    amount = 2000
17    discount = amount * 0.10
18    final_amount = amount - discount
19    final_amount = calculate_final_amount(amount, discount_rate)
20    print("Final Amount:", final_amount)

```

REFACTORING PYTHON FOR RECTANGLE AREA AND PERIMETER

Examine the Python code and identify repeated inline calculations related to discount and final amount. Refactor the program by creating a reusable function that calculates the final amount after a 10% discount. Improve readability and documentation.

AIAC_13.py

✓ Modifying Python file

Excellent progress: the discount code is exactly as described and ready to refactor. Next I'm editing AIAC_13.py to a helper function with docs and preserving output.

> Working...

Edited AIAC_13.py +0 -0

Evaluating...


```
exists = False
```

```
for student in students:
```

```
if name == student:
```

```
exists = True
```

```
if exists:
```

```
print(name, "found")
```

```
else:
```

```
print(name, "not found")
```

Expected Output:

An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.

Prompt Used: Analyze the Python code that checks whether names exist in a list using nested loops. Suggest a more efficient approach using appropriate data structures and refactor the code while keeping the same output.

The screenshot shows a code editor with a Python script named `AIAC_13.py`. The script contains two versions of a name-checking function. The first version (lines 1-11) uses nested loops to check if a name from `check_list` exists in `students`. The second version (lines 12-13) uses a `set` for `students` to optimize the lookup. A sidebar on the right, titled "OPTIMIZING NAME CHECK IN PYTHON CODE", provides a tip about using sets for O(1) lookups and shows the refactored code. A status bar at the bottom indicates that the refactored solution was applied, updating the code file and editing `AIAC_13.py` with +8 and -8 lines.

```
1 students = ["Anil", "Bhavya", "Charan", "Divya"]
2 check_list = ["Charan", "Esha", "Bhavya"]
3
4 # Use a set for faster membership tests (average O(1) instead of O(n))
5 student_set = set(students)
6
7 for name in check_list:
8     exists = False
9     for student in students:
10         if name == student:
11             exists = True
12             if exists:
13                 print(name, "found")
14             else:
15                 print(name, "not found")
16
17 if name in student_set:
18     print(name, "found")
19 else:
20     print(name, "not found")
```

Tip: You can undo Copilot's changes to any point by clicking Restore Checkpoint.

Analyze the Python code that checks whether names exist in a list using nested loops. Suggest a more efficient approach using appropriate data structures and refactor the code while keeping the same output.

✓ Updating code file

Great news: I found the exact code and the optimization opportunity in one pass. Next I'm applying a leaner, faster version using a set for O(1) lookups while preserving behavior and output.

✓ Edited AIAC_13.py +8 -8

✓ Refactored solution applied

```
AIAC_13.py > ...
1  students = ["Anil", "Bhavya", "Charan", "Divya"]
2  check_list = ["Charan", "Esha", "Bhavya"]
3  |
4  # Use a set for faster membership tests (average O(1) instead of O(n))
5  student_set = set(students)
6
7  for name in check_list:
8      if name in student_set:
9          print(name, "found")
10     else:
11         print(name, "not found")
12
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [] [X] ... | [] [X]

```
PS C:\Users\Sai Architha\OneDrive\Desktop\AIAC> & "C:/Users/Sai Architha/AppData/Local/Python/pythoncore-3.14-64/python.exe" "c:/Users/Sai Architha/OneDrive/Desktop/AIAC/AIAC_13.py"
Charan found
Esha not found
Bhavya found
```

Explanation: The nested loop was inefficient because it compared each name multiple times. By converting the students list into a set, the lookup becomes faster. This improves performance and simplifies the logic.

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

Instructions:

- Detect hardcoded numeric values used directly in expressions.
- Define descriptive constants at the beginning of the program.
- Update calculations to use these constants without changing results.

Sample Code:

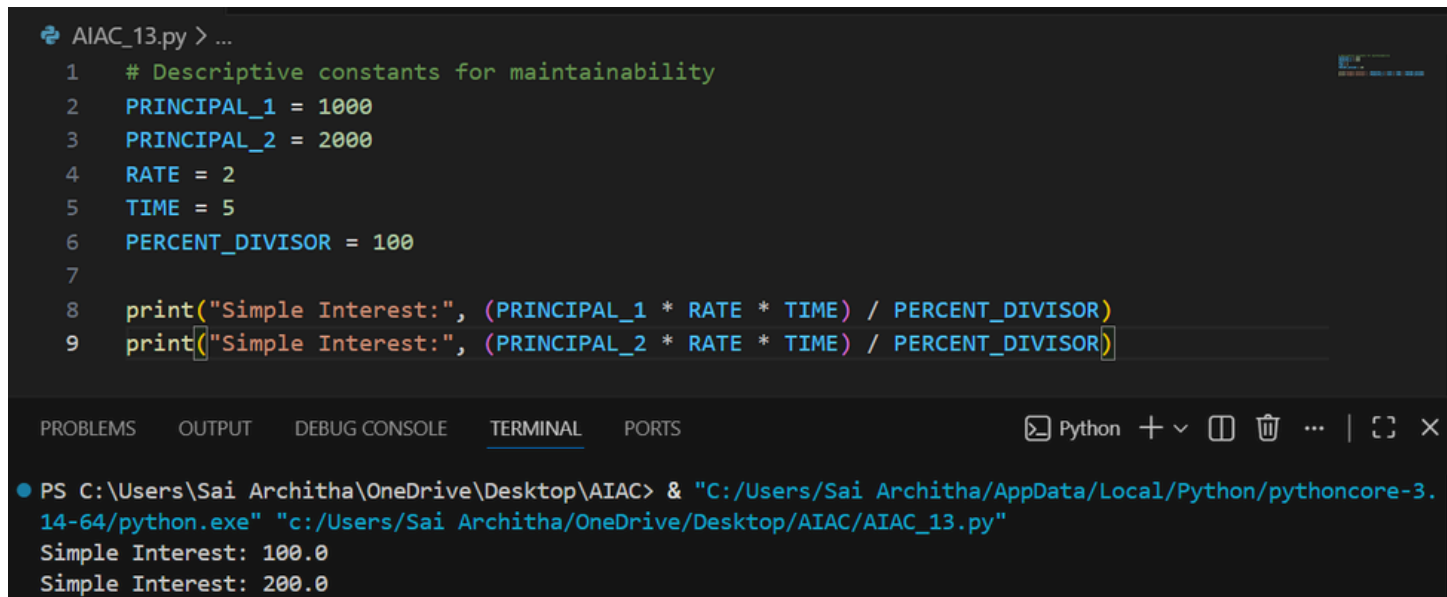
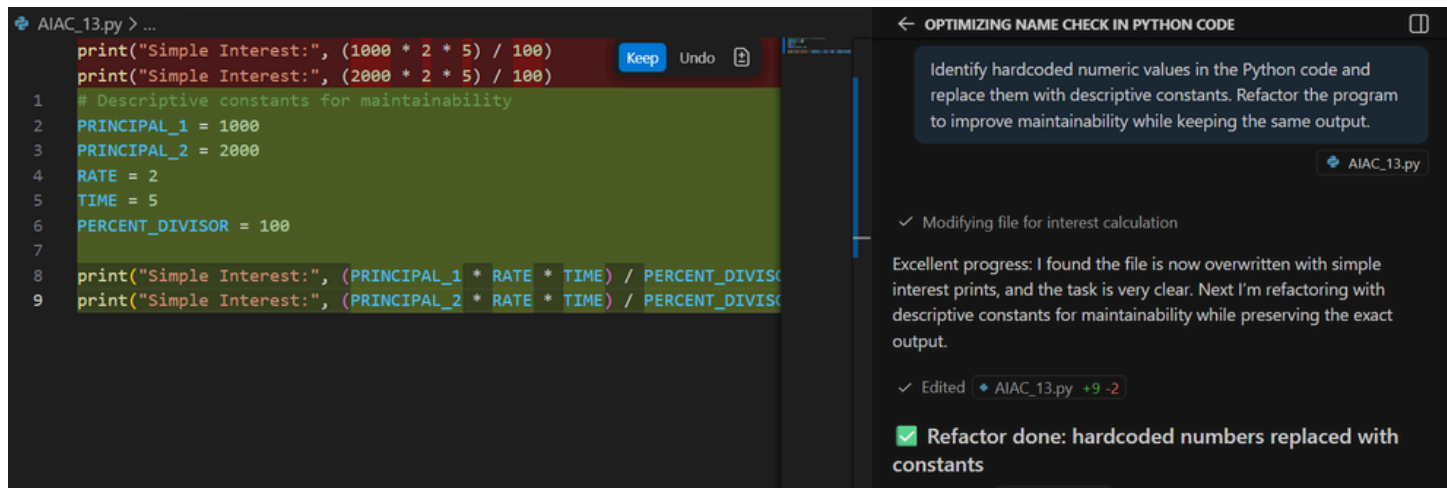
```
print("Simple Interest:", (1000 * 2 * 5) / 100)
```

```
print("Simple Interest:", (2000 * 2 * 5) / 100)
```

Expected Output:

A revised Python program using constants such as RATE and TIME for clearer and maintainable calculations.

Prompt Used: Identify hardcoded numeric values in the Python code and replace them with descriptive constants. Refactor the program to improve maintainability while keeping the same output.



Explanation: Hardcoded values such as interest rate and time were replaced with constants (RATE, TIME). This makes the program easier to update and improves readability.

Task Description -5 (Refactoring – Enhancing Variable Naming and Code Clarity)

Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments.

Instructions:

- Replace non-descriptive variable names with meaningful identifiers.

- Add inline comments where the logic may be unclear.
- Ensure program output remains the same.

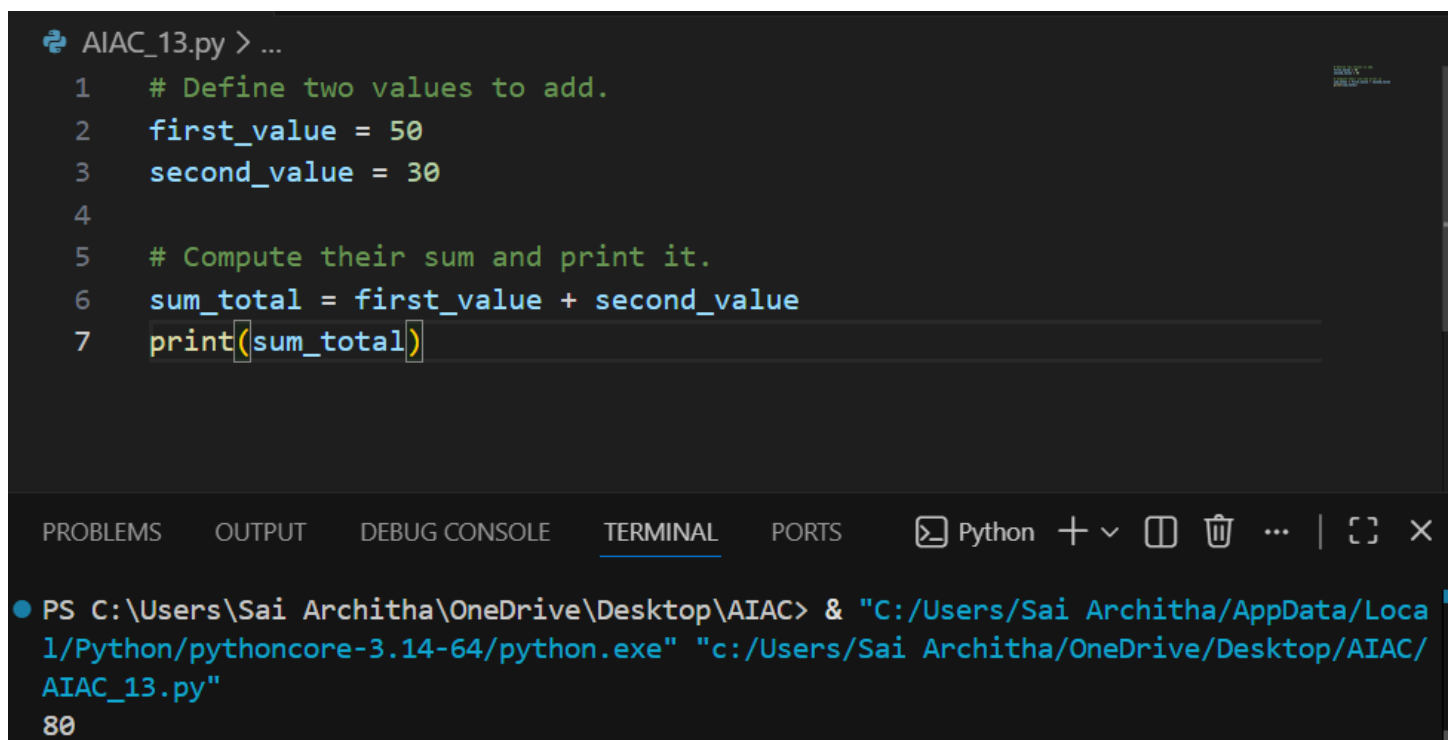
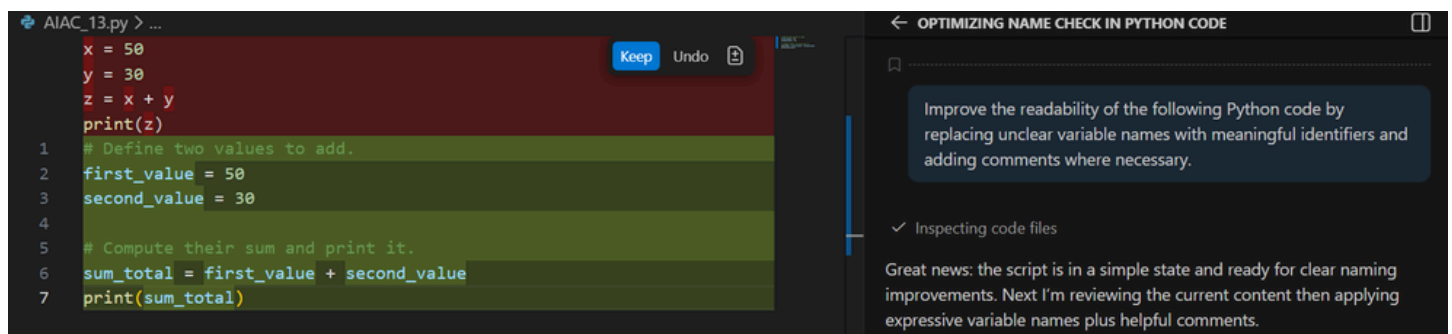
Sample Code:

```
x = 50  
  
y = 30  
  
z = x + y  
  
print(z)
```

Expected Output:

A Python program with clear variable names and explanatory comments that improve readability without altering functionality.

Prompt Used: Improve the readability of the following Python code by replacing unclear variable names with meaningful identifiers and adding comments where necessary.



Explanation: The variables x, y, and z were replaced with meaningful names (num1, num2, total_sum). Comments were added to explain each step, improving readability.